BriteTest

Runner Internal Guide

Document Version: 1.0.0
Runner Version: 1.0.0
Test Version: 1.0.0

This guide documents the internal architecture and design of the Runner Famework and API. It complements the Runner Internal Reference.

Preface

This document is intended for contributors who need guidance for the internals of the Runner Framework and API.

For a list of other documents and the repository layout, see the Documentation Guide (`Documentation_Guide.md`).

For a glossary of terms, see the Glossary Reference (`Glossary_Reference.md`).

A printer-friendly PDF file for this document is available in `docs/pdf/`.

Document Version History

Document	Runner	Test	Date	Comment	Author/Editor
1.0.0	1.0.0	1.0.0	2026-06-11	Initial version.	Paul Sinclair

- The **Document** column records the document's version.
- The **Runner** column records the Runner API version current at the time this version of the document was published.
- The **Test** column records the Test API version current at the time this version of the document was published.

A version has the format `M.m.p` (Major, minor, patch) where `M` is the major version, `m` is the minor version, and `p` is the patch version. `p` increments when the document is updated without a change to `M` or `m`, and resets to 0 when `M` or `m` increases. The first table entry is the most recent version for this document at the time this document was published.

Table of Contents

Introduction 1
Public and Internal Name Conventions .... ??
Glossary 14

1. Introduction

Basic concepts and high-level description of the framework internals and design goals.

In this document, a *symbol* refers to any named entity in the framework, including:

- typedefs
- structs
- enums and enum values
- macros
- variables
- functions

Some internal symbols in `runnerapi.h` are exposed because the API macros expand into code that depends on internal types and helper functions. These symbols must be visible to user code for the macros to compile correctly, but they are not part of the public API and should not be used directly.

Their presence in the header reflects C implementation requirements, not intended usage. Contributors may modify these symbols as needed, provided the public API contract remains intact.

1.1. Public and Internal Naming Conventions

Any Runner API names that are public and visible to Runner API users are prefixed with `ra_...` (typically lowercase) or `RA_...` (typically uppercase).

Any framework names that are internal (but technically visible to Bunner API users) have the prefix `ra_internal_` or `RA_INTERNAL_`. These names should not be referenced by Runner API users.

In general, users of the API should not define names prefixed with `ra_` or `RA_`, `, or reference names prefixed with`ra_internal_or`RA_INTERNAL_‘.

2. Architecture Overview

- Process model
- Thread model
- Signal handling
- Fault detection
- Execution flow

3. Orchestrator Internals

- Initialization
- Argument parsing
- Report lifecycle
- Category aggregation

4. Test Group Internals

- Group initialization
- Execution scheduling
- Result accumulation

5. Test Execution Engine

- Expression evaluation
- Isolation modes
- Concurrency model
- Error and fault propagation

6. Process-Isolated Execution

- Child process creation
- Monitoring and timeouts
- Exit code interpretation
- Fault mapping

7. Thread-Isolated Execution

- Thread creation
- Synchronization
- Fault boundaries

8. Signal Guard System

- Installed handlers
- Supported signals
- Fault classification
- Recovery behavior

9. Report Generation Internals

- Output formatting
- Category totals
- Fault messages
- Notes and metadata

10. Internal State and Global Variables

(Placeholder for internal state descriptions.)

11. Error Handling and Safety Guarantees

(Placeholder for internal error semantics.)

12. Implementation Notes

- Portability considerations
- POSIX dependencies
- Platform differences

13. Future Improvements

(Placeholder for roadmap items.)

Glossary

For general terms used the documentation, see the Glossary document.

Runner-Specific Terms:

- **Orchestrator Lifecycle**: The sequence of initialization, group execution, test execution, and report finalization performed by the BriteTest framework.
- **Guard Behavior**: The mechanism BriteTest uses to catch runtime faults (e.g., segmentation faults) and continue executing remaining tests.
- **Isolation Semantics**: The rules governing how tests and groups run in threads or processes to prevent interference and ensure fault containment.
- **Concurrency Model**: The framework's rules for running tests concurrently within `RA_BEGIN_CONCURRENT` / `RA_END_CONCURRENT` blocks.
- **Execution Phases**: The internal stages of orchestrator operation, including initialization, argument parsing, group dispatch, test dispatch, and report writing.
- **Fault Handling Model**: The framework's strategy for capturing and reporting faults without terminating the entire test run.
- **Control File Promotion**: The process of replacing an outdated control file with a newly generated file when differences are expected or intentional.
- **Nested Group Behavior**: The rules governing how groups may contain other groups and how isolation and concurrency propagate through nested structures.
- **Concurrent Block Behavior**: The semantics of executing multiple tests in parallel within a concurrent block, including ordering and isolation rules.